This test will likely give you false assurances. If the underlying cache is misbehaving or was never implemented, the calculation will just run again and produce a new word list in the same order. Because both arrays have the same contents, your test will incorrectly pass.

Instead, you'd like to know whether or not first_try and second_try are actually referring to the same underlying object—not just two copies with identical contents.

For this stricter comparison, you'd use RSpec's equal matcher, which hands off to Ruby's equal? method behind the scenes:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
expect(second_try).to equal(first_try)
```

RSpec's own internal specs for the RSpec.configuration method use this technique to make sure that the method always returns the same RSpec::Core::Configuration instance:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
RSpec.describe RSpec do
  describe '.configuration' do
    it 'returns the same object every time' do
      expect(RSpec.configuration).to equal(RSpec.configuration)
    end
  end
end
```

If you prefer, you can also use be(x) as an alias for equal(x), to emphasize that this matcher is about *identity* rather than *value equality*:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
expect(RSpec.configuration).to be(RSpec.configuration)
```

The third notion of equality sits between these two in terms of strictness.

Hash Key Equality

Programmers rarely check hash key equality directly. As the name implies, it's used to check that two values should be considered the same Hash key. If you encounter a use of this method in the wild, it will likely be called from an object designed to behave like a dictionary.

RSpec's eql matcher, based on Ruby's built-in eql? method, checks for hash key equality. Generally, it behaves the same as the eq matcher (since eql? generally behaves the same as ==). One notable difference is that eql? always considers integers and floating-point numbers to be different: